Monitoramento da Qualidade do Ar

Integração MongoDB e Redis com Python - Etapas I e II

Bancos de Dados e Novas Tecnologias - Unochapecó

Professora: Monica Tissiani De Toni Pereira

Ariel Felipe Marques dos Santos; Everton Marlon Berlezi; Gustavo Desordi; Luiz Eduardo Neckel Ghilardi.

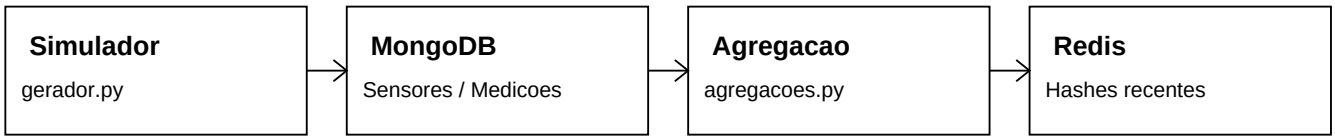
Chapecó, 29 de outubro de 2026.

1. Objetivo e divisão do grupo

O protótipo simula sensores ambientais, mantém o histórico no MongoDB, calcula resumos horários e grava os últimos resultados em Hashes no Redis. A demonstração é feita no console, sem interface gráfica.

Ariel: modelagem, cadastro e consultas (banco.py e consultas.py). Everton: simulador (gerador.py). Gustavo: agregações (agregacoes.py). Luiz: Redis e integração (redis_service.py e main.py). O script testes.py reúne os testes da Etapa II.

2. Arquitetura e organização



O Python gera os dados e os grava no MongoDB. O módulo de agregação solicita ao MongoDB o cálculo dos resumos. O Python seleciona a janela mais recente de cada entidade e usa HSET para gravá-la no Redis. HGETALL consulta os campos do Hash.

O pacote contém codigo/ com somente arquivos Python e documentacao/ com este relatório. README.md e requirements.txt ficam na raiz. O README identifica os quatro integrantes, explica a execução no Linux

3. Modelagem e implementação

O banco QualidadeAr tem as coleções Sensores e Medicoes. Sensores guarda _id, modelo, descrição, localização, ativo e grandezas. Medicoes guarda sensor_id, timestamp UTC, pm25, pm10, co2, temperatura e umidade. O sensor_id corresponde ao _id do cadastro. O índice {sensor_id: 1, timestamp: -1} ajuda nas consultas por sensor e período; unique=True impede repetições do mesmo sensor/horário.

São dois sensores por cidade: Chapecó, Xanxerê e Concórdia. Cada sensor gera 288 leituras, a cada cinco minutos, de 20/10/2026 00:00 UTC até 21/10/2026 00:00 UTC, excluindo o fim. No total, são 1.728 documentos. Os dados e coordenadas são ilustrativos. ReplaceOne com upsert e identificadores fixos permite repetir a carga sem duplicação.

4. Agregações e Redis

A consulta usa o intervalo [início, fim): inclui o início e exclui o fim. O pipeline aplica $match para período e valores numéricos; $lookup e $unwind para obter a cidade do sensor; $group para reunir ano, mês, dia e hora; $sum para contagem; $avg para as cinco médias; $max para máximos de PM2.5 e PM10; e $project para selecionar os campos. O Python monta inicio_periodo e fim_periodo.

A média por cidade usa as leituras originais, e não a média simples das médias dos sensores. Os resultados são armazenados em qualidadear:sensor:S001 e qualidadear:cidade:SC:chapeco, por exemplo. A cidade é normalizada sem acentos e em minúsculas. São seis Hashes de sensores e três de cidades, cada um com o resumo de maior fim_periodo.

5. Execução e resultados

A execução padrão utiliza Linux, Python com PyMongo e redis-py, MongoDB em localhost:27017 e Redis em localhost:6379. Os serviços devem estar previamente instalados e ativos. Na raiz do projeto:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python codigo/main.py
python codigo/testes.py
```

Em 29/10/2026, os dois programas foram executados com MongoDB e Redis reais nas portas padrão. A máquina disponível era macOS, com Python 3.9.6, MongoDB 8.0.32, Redis 8.10.2, PyMongo 4.18.3 e redis-py 7.0.1.

```
Sensores: 6 em 3 cidades
Medicoes geradas e armazenadas: 1728
Consulta S001, das 10h as 11h UTC: 12 leituras
Resumos por sensor: 144
Resumos por cidade: 72
Hashes gravados: 9
```

A aplicação foi executada duas vezes: a contagem permaneceu em 1.728. Os 216 resumos e os nove Hashes foram comparados com um cálculo independente em Python, com resultados coincidentes.

6. Testes com dados diferentes - Etapa II

O script testes.py utiliza um banco de dados separado, denominado QualidadeArTesteGerador, preservando os dados da base principal QualidadeAr. São cadastrados seis sensores em três cidades e geradas 1.728 medições fictícias, utilizando uma seed diferente (30) e um período de 24 horas em janeiro de 2030, com intervalos de cinco minutos entre as leituras.

O script verifica as consultas por sensor e período, as agregações por sensor e cidade, as cinco médias, os dois valores máximos e as quantidades de medições. Também testa a integração com Redis, comparando os resumos mais recentes armazenados nos Hashes com os resultados do MongoDB, além de verificar se a repetição da inserção não gera documentos duplicados. Os dados permanecem disponíveis para consulta no MongoDB Compass e podem ser removidos posteriormente por meio da opção --limpar, sem alterar a base principal.

7. Dificuldades e soluções

A versão inicial tinha recursos além do solicitado. A revisão manteve os seis módulos simples, adotou os serviços padrão e colocou Linux como ambiente principal. A Etapa II foi atendida no pacote com a separação das pastas, o diagrama e um script de teste com outros dados.

As conexões locais tinham sido bloqueadas durante a preparação anterior. Nesta revisão, o acesso aos serviços funcionou, permitindo executar o protótipo e os testes reais. O agrupamento usa ano, mês, dia e hora para evitar depender de $dateTrunc e $dateAdd.

8. Ferramentas utilizadas e inteligência artificial

Ferramentas: Python, PyMongo, redis-py, MongoDB, Redis, terminal e MongoDB Compass. ChatGPT/Codex auxiliou na escrita, revisão, organização, testes e documentação. Claude foi usado na revisão apresentada pelo grupo, que apontou excessos e problemas de portabilidade. O código está dividido conforme os assuntos da apresentação.

9. Apresentação e entrega no GitHub

A apresentação começa com uma introdução ao problema e à arquitetura. Depois, cada integrante demonstra sua parte no console e o grupo executa codigo/testes.py, mostrando os dados diferentes e as verificações aprovadas. O README descreve a ordem e os comandos.

10. Referências

PEREIRA, Monica Tissiani De Toni. Estudo de Caso: Monitoramento da Qualidade do Ar -Integração MongoDB + Redis com Python. Trabalhos práticos, Etapas I e II. Chapecó: Unochapecó, 2026.

Divisão das tarefas do grupo. Documento fornecido pelos integrantes, 2026.

MONGODB. Aggregation (PyMongo). https://www.mongodb.com/docs/languages/python/pymongodriver/current/aggregation/. Consulta na elaboração: 22 out. 2026.

REDIS. HSET e HGETALL. https://redis.io/docs/latest/commands/hset/ e https://redis.io/docs/latest/commands/hgetall/. Consulta na elaboração: 22 out. 2026.

PYTHON SOFTWARE FOUNDATION. datetime. https://docs.python.org/3/library/datetime.html. Consulta na elaboração: 22 out. 2026.

OPENAI. ChatGPT/Codex. https://chatgpt.com/overview/. ANTHROPIC. Claude. https://claude.ai/. Ferramentas utilizadas na elaboração e revisão.

Código-fonte: os sete arquivos Python e requirements.txt acompanham o relatório no pacote e devem ser enviados ao repositório do grupo.